El problema: un asistente sin contexto previo

Turno 1

Usuario: «Me llamo Alex, trabajo con Python»

Asistente: «¡Hola! ¿En qué te ayudo?»

Turno 2 · mismo chat

Usuario: «¿Cómo puedo leer un archivo CSV?»

Asistente: «¿Qué es un CSV?» × no sabe quién eres

¿Qué ha pasado aquí?

- El LLM no guarda nada entre llamadas (stateless)
- Tu aplicación tampoco reenvía el historial
- Cada turno parte de cero: el contexto se pierde
- La solución: guardar y reenviar el historial — eso es la memoria

Sin contexto guardado no hay continuidad: lo tiene que aportar la aplicación.

Memoria conversacional: qué es y qué NO es

SÍ es memoria conversacional

- Guardar los mensajes de la conversación
- Reenviarlos como contexto en el siguiente prompt
- Un mecanismo de TU aplicación: checkpointer + hilo
- Alcance: una sesión concreta — y solo esa
- Algo que puedes inspeccionar y borrar

NO es…

- Que «el modelo recuerde» por sí solo
- La base de conocimiento del producto
- RAG, embeddings ni bases vectoriales (otra píldora)
- Almacenamiento permanente y silencioso
- Sinónimo de buena respuesta: más contexto no siempre es mejor

La memoria la construye tu aplicación, no el LLM.

Historial, memoria a corto plazo y memoria persistente

1 · Historial conversacional

- La lista de mensajes (usuario / IA) de la conversación actual
- Ej.: «Me llamo Alex…» + respuesta
- Es lo que se reenvía al modelo en cada turno
- Solo existe mientras la app lo conserva

2 · Memoria a corto plazo

- Sesión o hilo: checkpointer + thread_id
- Se reanuda al continuar la conversación
- LangChain: InMemorySaver (demo) · Postgres (producción)
- Muere al cerrar o borrar el hilo

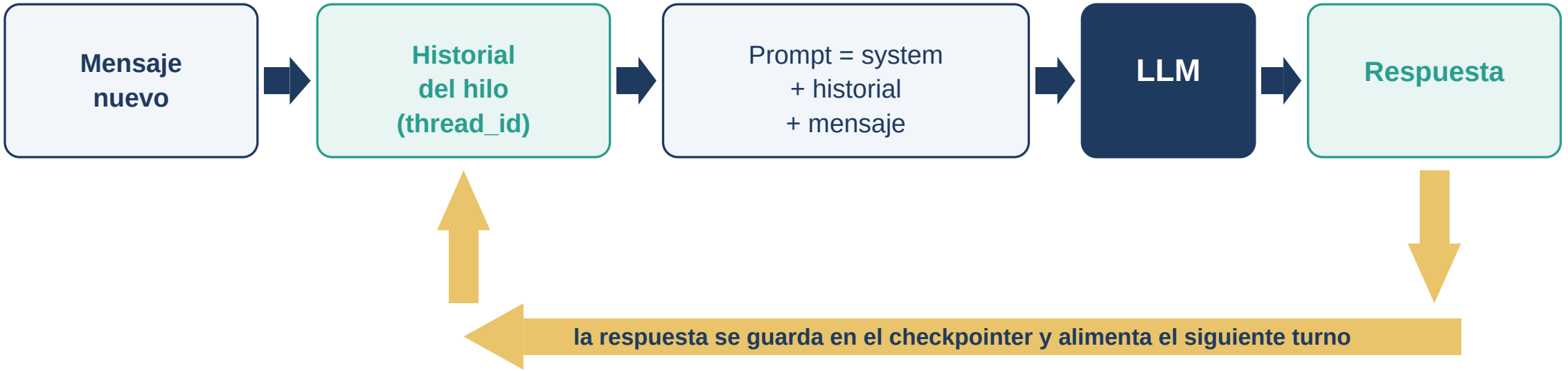
3 · Memoria persistente

- Cruza sesiones: preferencias y hechos del usuario
- LangChain: stores con namespaces · LlamaIndex: Memory Blocks
- Exige consentimiento, retención y borrado
- No la usamos en esta píldora: es otro nivel de diseño

Hoy trabajamos los niveles 1 y 2: el historial y la memoria de la sesión (hilos).

El flujo de memoria de una aplicación

Un turno cualquiera, paso a paso:



Sesión nueva = otro thread_id = historial vacío = sin contexto previo.

En la demo: checkpointer = InMemorySaver() · un thread_id por conversación

Demo con LangChain y comparación con LlamaIndex

LangChain — la demo que vas a ver

```
agente = create_agent(modelo, checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "sesion-alex"}}
agente.invoke({"messages": [msg]}, cfg)   # turnos 1 y 2
agente.get_state(cfg)                     # inspección
checkpointer.delete_thread("sesion-alex")  # borrado
```

- **Mismo thread_id → continuidad entre turnos**
- Otro thread_id → sesión aislada (sin datos del otro)
- get_state() → historial completo · delete_thread() → borrado selectivo

LlamaIndex — comparación (1 min)

- Objeto central: Memory con session_id
- Corto plazo: cola FIFO de ChatMessage; al llegar al límite se vacía (flush)
- Largo plazo: Memory Blocks — estáticos, hechos extraídos, vectoriales
- ChatMemoryBuffer está obsoleto: usar Memory
- Misma idea: alguien guarda el historial y lo reenvía

Solo se implementa LangChain: LlamaIndex es comparación conceptual.

Límites, privacidad y conclusiones

Límites y riesgos

- El historial crece: más tokens → más coste y peor contexto
- La ventana de contexto tiene techo: hay que podar o resumir
- Datos sensibles o erróneos se reutilizan como si fueran ciertos
- Un thread_id compartido = fuga de contexto entre usuarios
- Recuperar mal = respuestas mal, con apariencia de certeza

Conclusiones

- **1. Sin historial reenviado, no hay continuidad**
- **2. Corto plazo = hilos aislados (thread_id + checkpointer)**
- **3. Persistente = entre sesiones: solo con diseño y consentimiento**
- **4. Minimiza · aísla · borra: guardar más no es mejor**

Criterio ético: docs/criterio_etico.md · corrección del reto: 4 tests = 4 criterios